Table des matières

Cours 12 — Capstone : un pipeline de bout en bout 1
Objectifs 2
1. Ce que « fini » veut dire 2
2. La carte de Boréal 3
3. Ce qui se versionne, ce qui se reconstruit 5
4. Orchestrer une reconstruction 6
5. Trois preuves 8
6. Le chemin chaud de bout en bout 9
7. Le tableau de bord, livrable versionné 9
8. L'incident vu de l'amont 10
9. La passation 12
Points clés à retenir 13
Exercices 14

Cours 12 — Capstone : un pipeline de bout en bout

Théorie · Module 12/12 — Formation Data Engineering · avancé

Où vous êtes : source → **ingestion** → **lac** → **entrepôt** → **restitution**. Les onze premiers cours ont construit chaque maillon de la chaîne de Véhicules Boréal inc., un à la fois. Celui-ci n'en ajoute presque aucun. Il prend la plateforme entière, l'efface, la refait à partir du dépôt, prouve qu'elle est revenue identique, puis la passe à ceux qui la liront.

En 30 secondes :

- Un pipeline qui tourne n'est pas fini. Il est fini quand il se refait de zéro à partir du dépôt, qu'une preuve dit qu'il est revenu identique, et qu'une autre personne peut le reprendre.
- Ce qui se versionne, c'est ce qu'on écrit : le code, les contrats, les DAG, le tableau de bord. Tout le reste se refait : le lac, l'entrepôt, les séries, les verdicts.
- Un seul DAG, sans horaire, refait tout le chemin froid de Boréal en un peu plus de trois minutes. Il a fait tomber trois défauts qu'aucun cours n'avait vus, dont un contrat qui jugeait autrement dans le conteneur, parce que le conteneur compte en UTC.

© 2026 iAlexMG. Tous droits réservés.

- Refaire n’est pas prouver. Trois preuves s’empilent : les comptes attendus, l’égalité des lignes dans les deux sens, et les contrats. Toutes rendent 0 écart.
- Une reconstruction refait un état, jamais une histoire : le journal d’avant raconte deux livraisons que les sources ne portent plus.
- L’incident de l’automne 2025 se lit maintenant de bout en bout, de la pression instable du 20 octobre à la plage des 229 motoneiges que la `RAW` livre à Analytics Engineering.

L’idée du cours. Une plateforme qu’on n’ose pas effacer n’appartient à personne : elle appartient à l’état de la machine où elle tourne, et au souvenir de celui qui l’a montée. Le jour où cette machine casse, ou où cette personne part, on découvre ce qui n’était écrit nulle part. Le capstone fait ce jour-là exprès, dans de bonnes conditions, pendant qu’on peut encore comparer.

Le cours compte deux séances. La première, sections 1 à 5, tient le chemin froid : ce qui se refait, le DAG qui le refait, et les preuves. La seconde, sections 6 à 9, tient le chemin chaud, le tableau de bord, l’incident et la passation.

Objectifs

- Dire ce qui distingue une plateforme finie d’une plateforme qui tourne.
- Situer chaque maillon de la chaîne de Boréal, et le cours qui l’a construit.
- Séparer ce qui se versionne de ce qui se reconstruit, et dire ce qu’une reconstruction ne refait jamais.
- Orchestrer une reconstruction : un DAG sans horaire, ses dépendances de données, sa ressource partagée.
- Reconnaître trois défauts qui n’apparaissent qu’en changeant de machine : un fichier attaché deux fois, un pilote absent de l’image, un fuseau horaire différent.
- Prouver une reconstruction par les comptes, par les lignes et par les contrats.
- Rejouer le chemin chaud de zéro, et dire pourquoi l’alerte revient à la même minute.
- Livrer un tableau de bord comme un fichier du dépôt, et dire où s’arrête le travail de Data Engineering.
- Remonter un incident depuis l’amont, et nommer ce que la `RAW` doit porter pour qu’on l’analyse.
- Passer une plateforme : le `README.md` du dépôt, `LIVRAISON.md`, les contrats, le kit, une étiquette.

---

1. Ce que « fini » veut dire

Labo : `Labo_12_DE.md`, étapes 4 et 5.

© 2026 iAlexMG. Tous droits réservés.

Depuis le Cours 03, chaque labo a laissé la plateforme un peu plus riche, et jamais on ne l'a effacée. L'entrepôt que vous avez sous les yeux porte donc onze cours d'histoire : des chargements par tranches, des rattrapages, des essais, des verdicts. Il est juste, les contrats l'ont montré au Cours 11. Mais personne ne sait s'il est reproductible, et c'est une autre question.

Une plateforme finie tient trois promesses, et aucune ne se voit tant qu'on ne l'a pas mise à l'épreuve.

Promesse	Ce qu'elle veut dire	L'épreuve du capstone
Elle se refait	le dépôt, les sources et la plateforme suffisent : rien ne dépend d'un geste fait une fois à la main	effacer le lac et l'entrepôt, puis tout relancer par un seul DAG
Elle se prouve	ce qui revient égale ce qui était là, et chaque contrat rend son attendu	les comptes, les lignes dans les deux sens, les contrats
Elle se passe	une autre personne la reprend sans vous : elle sait quoi lancer, dans quel ordre, et quoi attendre	le `README.md` du dépôt, `LIVRAISON.md`, les contrats, une étiquette `v1.0`

La première est la plus exigeante, et la seule qui trouve des défauts. Tant qu'un pipeline tourne sur la machine où on l'a écrit, un geste oublié ne se voit pas : un dossier créé à la main, un paquet installé un soir, une variable posée dans un terminal. Effacer et refaire, c'est rejouer tous ces gestes d'un coup, par écrit, et voir lesquels manquaient. Le Cours 06 l'avait dit du rattrapage : ce qui se rejoue se prouve. Le capstone l'applique à la plateforme entière.

« De zéro » ne veut pas dire « de rien ». Les sources restent, et c'est ce qui rend l'épreuve possible : chez Boréal, `sources/` joue le rôle des livraisons gardées telles qu'elles sont arrivées. Une entreprise qui jette ses livraisons après les avoir chargées ne peut plus rien reconstruire ; elle ne peut que restaurer une sauvegarde, et espérer qu'elle est bonne. La plateforme reste aussi, avec ses services : elle appartient à l'équipe qui l'exploite (Cours 04), et le capstone ne refait que ce que Data Engineering produit.

2. La carte de Boréal

Onze cours ont posé onze morceaux. Mis bout à bout, ils forment deux chemins qui partent des mêmes systèmes et ne se rejoignent qu'à l'usine : le chemin froid, qui charge par lots ce qui a été livré, et le chemin chaud, qui écoute ce qui arrive (Cours 09).

Le chemin froid : sources → bronze → `RAW` et argent → contrats → Analytics Engineering.

Maillon	Cours	Ce qu'il fait	Ce qui le prouve
Le bronze	02	l'historien et la télémétrie, du CSV et du JSON au Parquet partitionné par jour	chaque script compare la source et ce qu'il a écrit

© 2026 iAlexMG. Tous droits réservés.

Maillon	Cours	Ce qu'il fait	Ce qui le prouve
La `RAW`	03	22 livraisons en 22 tables, en texte, avec leur lignage et un journal des chargements	un second passage ne charge rien
La base et l'API	04	l'ERP par ODBC, la météo par l'API d'ECCC	l'état tiré des extraits égale la base, 0 écart dans les deux sens
Le pipeline	05	une porte d'entrée, une configuration, un journal, des reprises, des tests	`pytest`, et `pipeline.executions`
L'ordonnanceur	06	les DAG quotidien et mensuel, la fenêtre, le pool	le rattrapage égale le chargement d'un bloc
Les séries de l'usine	08	l'heure résolue, la moyenne pondérée, la couverture, le schéma `argent`	la base de séries temporelles rend la même série, 0 écart
Les tables ouvertes	10	la télémétrie et l'argent en Iceberg, la même télémétrie en Delta	le voyage dans le temps rend l'état au 2025-12-31
Les contrats	11	quatre contrats, leurs verdicts, leur place dans les DAG	chaque contrat attrape son événement

Le chemin chaud : courtier MQTT → pont → Kafka → base de séries temporelles → alerte et tableau de bord.

Maillon	Cours	Ce qu'il fait	Ce qui le prouve
Le courtier	09	l'usine et les véhicules publient, l'abonné écrit l'archive	l'abonné égale l'archive, 0 écart dans les deux sens
Kafka	09	le journal qu'on garde, par sujet, par partition, par clé	un second groupe relit tout en moins d'une seconde
La base du temps réel	08, 09	le schéma `temps_reel`, dont la clé primaire écarte les doublons	62 726 points, 0 perdu

© 2026 iAlexMG. Tous droits réservés.

Maillon	Cours	Ce qu'il fait	Ce qui le prouve
L'alerte	09	la vibration du compresseur 2, en fenêtres de 15 minutes sur le temps de l'événement	levée le 2025-10-22 à 8 h 15
Le tableau de bord	12	Grafana, branché sur `temps_reel`	ce cours

Un cours manque aux deux tableaux, et ce n'est pas un oubli : Spark, au Cours 07, a fait le même travail que DuckDB, avec les mêmes réponses, vingt à soixante fois plus lentement. Il n'entre dans aucun chemin de Boréal. Choisir de ne pas s'en servir, mesures à l'appui, est un résultat du parcours au même titre que ce qu'on garde.

3. Ce qui se versionne, ce qui se reconstruit

Labo : `Labo_12_DE.md`, étapes 4 et 10.

Tout ce qui existe dans `Projet_Boreal` tombe dans une de trois familles, et la frontière entre elles décide de ce qu'on peut effacer.

Ce qui existe	Où	Versionné	Revient par
le code du pipeline, les scripts SQL, les DAG	`01_de/`	oui, dans votre dépôt	Git
les contrats, le tableau de bord, les post-mortems	`01_de/contrats/`, `grafana/`, `postmortems/`	oui	Git
la plateforme : le compose, l'image d'Airflow, les réglages de Grafana	`plateforme/`	oui, par l'équipe de la plateforme, dans `_univers/plateforme/`	une copie, puis `docker compose build` et `up -d`
les secrets	les deux `.env`	jamais	à la main, depuis leur modèle `.env.exemple`
les sources	`sources/`	non : ce sont les livraisons des systèmes de Boréal	elles se reçoivent ; ici, le générateur
le lac bronze et argent, l'entrepôt	`plateforme/lac/`, `plateforme/entrepot/`	non	le DAG de reconstruction
les tables Iceberg et Delta	`plateforme/lac/iceberg/`, `plateforme/lac/delta/`	non	les huit commandes du Cours 10

© 2026 iAlexMG. Tous droits réservés.

Ce qui existe	Où	Versionné	Revient par
les séries dans TimescaleDB, le temps réel	volume du service `series`	non	`series_usine.sql` ; le chemin chaud, par un rejeu
les verdicts et les exécutions	`pipeline.controles`, `pipeline.executions`	non	ils se rejugent à tout moment

La première famille s'écrit, et c'est la seule qui entre dans votre dépôt. La deuxième s'écrit aussi, mais par d'autres : l'équipe de la plateforme, les systèmes sources, et vous-même pour les secrets, qui ne voyagent jamais dans Git (Cours 05). La troisième se dérive des deux premières : on peut l'effacer sans crainte, à condition de savoir la refaire, et le capstone vérifie justement qu'on le sait.

Une reconstruction refait un état, jamais une histoire. Le journal des chargements de votre entrepôt compte 1 706 lignes, et celui qu'on refait n'en comptera que 1 704. Les deux lignes de trop racontent les premières versions de deux fichiers livrés de nouveau sous le même nom au Cours 06 : l'export de la GMAO et le catalogue du corpus. Les sources n'en portent plus que la dernière, et aucune commande ne peut recharger un fichier qui n'existe plus. Il en va de même des exécutions passées, des verdicts d'hier et de l'heure de réception d'un message. Ce qui garde une histoire, c'est une sauvegarde, pas un dépôt. Une preuve honnête le dit, et compte à part ce qu'elle ne peut pas refaire, plutôt que de l'appeler écart ou de le cacher (section 5).

Piège — effacer ce qu'un autre tient. La base de séries temporelles monte en lecture seule le dossier des séries du lac, `plateforme/lac/argent/usine` (Cours 08). Effacé sous Windows pendant que ce service tourne, le lac disparaît pour tout le monde, sauf pour ce montage, qui retient son chemin : le conteneur d'Airflow ne peut plus recréer `plateforme/lac`, et le DAG échoue en une seconde sur `[Errno 17] File exists`. Avant d'effacer, on arrête ce qui monte ce qu'on efface, et on le rallume quand la reconstruction l'a refait.

4. Orchestrer une reconstruction

Labo : `Labo_12_DE.md`, étapes 1, 2, 3 et 5.

La reconstruction de Boréal tient dans un DAG, `boreal_reconstruction`, sans horaire : on le déclenche quand on veut prouver que le dépôt suffit, après une panne, un changement de machine, ou avant de livrer. Il ne rattrape rien et ne tourne qu'une fois à la fois.

Tâche	Ce qu'elle refait	Dépend de
`bronze`	l'historien et la télémétrie en Parquet	rien
`raw`	les 22 tables de la `RAW` et le journal des chargements	rien, mais après le bronze pour garder l'ordre du Cours 05
`corpus`	la documentation déposée dans le bronze	rien
`argent`	les séries de l'usine, dans l'entrepôt et le lac	le bronze de l'historien

© 2026 iAlexMG. Tous droits réservés.

Tâche	Ce qu'elle refait	Dépend de
`contrats`	les verdicts des quatre contrats, sur tout l'historique	tout ce qui précède
`erp`	la base de l'ERP lue par ODBC, la preuve du Cours 04	la base de l'ERP
`meteo`	la météo tirée de l'API d'ECCC	le réseau

Les flèches du DAG disent les dépendances des données, et rien d'autre : bronze → `raw` → corpus → argent → contrats. L'ERP et la météo ne nourrissent ni la `RAW` ni l'argent, et se posent à côté, sans flèche. Toutes les tâches prennent pourtant le pool `entrepot` du Cours 06, puisque toutes ouvrent le fichier : elles passent donc une à une, même celles qu'aucune flèche ne relie. Les deux branches ne font gagner aucun temps ; elles disent ce qui dépend de quoi, et c'est ce qu'un lecteur du DAG doit pouvoir lire.

Une seule tâche est neuve : `argent`. Le Cours 08 refaisait les séries par une commande à part, `duckdb -f 01_de/sql/argent_usine.sql`, et une reconstruction qui demande une commande à la main au milieu n'est pas une reconstruction. L'étape `argent` du pipeline lance ce même script, et elle a fait tomber le premier des trois défauts que le capstone a trouvés en changeant de machine.

Défaut	Ce qui se passe	La correction
Un fichier attaché deux fois	`argent_usine.sql` attache l'entrepôt, parce qu'on le lance depuis une base vide ; le pipeline tient déjà l'entrepôt ouvert, et DuckDB refuse : `Unique file handle conflict`	l'étape retire l'`ATTACH` et le préfixe `entrepot.` avant d'exécuter, et vérifie qu'elle a bien trouvé ce qu'elle retire ; le script reste lançable seul
Un pilote absent de l'image	l'étape `erp` lit la base par ODBC ; l'image d'Airflow n'a ni pilote ni `pyodbc`, et la base ne s'appelle pas `localhost:5433` dans le réseau des conteneurs	le pilote PostgreSQL de Debian et `pyodbc` posés dans l'image, à sa construction ; l'adresse `erp:5432` donnée par `PIPELINE_ERP_SERVEUR` et `PIPELINE_ERP_PORT`, la surcharge du Cours 05
Un fuseau horaire différent	la fenêtre du contrat de l'argent porte sur `heure`, un instant absolu ; comparé à une date, il se lit dans le fuseau de la session, celui de Boréal sous Windows, UTC dans le conteneur	la fenêtre posée sur `heure AT TIME ZONE 'America/Toronto'`, le jour de Boréal, quelle que soit la machine qui juge

Le troisième est le plus sournois, parce qu'il ne plante pas. Jugé sur avril 2026, un mois qui n'existe pas, le contrat rend sous Windows 30 jours manquants et une règle rompue. Dans le conteneur, il rend 29 jours manquants et deux règles rompues : les quatre dernières heures du 31 mars, de 20 h à minuit à l'heure de l'Est, sont déjà le 1er avril en UTC, et elles tombent dans la fenêtre d'avril, où leur production ne retombe plus sur le cumul de leur journée. Pire, le 31 mars jugé seul, comme le ferait un DAG quotidien, rompt dans le conteneur une règle bloquante que Windows juge respectée, alors que rien n'est faux dans les séries. Le même contrat, sur les mêmes données, rend deux verdicts selon la machine. La correction ne change rien de ce qu'un consommateur lit, et le contrat passe donc de

© 2026 iAlexMG. Tous droits réservés.

1.0.0 à 1.1.0, le chiffre du milieu (Cours 11). Il gagne dans la même version une règle qui manquait : `heures_du_jour_livrees`, la fraîcheur de l'argent. La version 1.0.0 ne disait rien d'un jour absent, et un mois entier aurait pu manquer sans qu'une règle rompe.

À retenir — une dépendance se pose à la construction de l'image, pas au démarrage. Le pilote ODBC est un paquet du système : il s'installe en `root`, dans le `Dockerfile.airflow`, et l'image rend ensuite la main à l'utilisateur `airflow`. La règle est celle de DuckDB au Cours 06 : ce qui s'installe à chaque démarrage finit par ne plus s'installer, un jour où le dépôt de paquets ne répond pas.

Reste le coût. Dans le conteneur, l'étape `argent` prend 105 secondes, contre 12 sous Windows : le script relit tout le bronze de l'historien à travers le montage de WSL 2, le coût de la traversée vu aux Cours 05 et 06. Le DAG entier refait la plateforme en un peu plus de trois minutes, dont plus de la moitié pour cette seule tâche. Sur un serveur Linux où les données vivent au même endroit que les conteneurs, cet écart disparaît.

5. Trois preuves

Labo : `Labo_12_DE.md`, étape 6.

Un DAG au vert dit que chaque tâche a rendu le code 0. Le Cours 11 a montré ce que vaut ce vert : un mois absent se charge sans erreur. Refaire n'est donc pas prouver, et trois preuves s'empilent, chacune plus exigeante que la précédente.

Preuve	Ce qu'elle compare	Ce qu'elle voit	Ce qu'elle ne voit pas
Les comptes	chaque table contre `LIVRAISON.md`, par le kit 02 et son option `--verifier`	une table absente, un fichier chargé deux fois, une convention rompue	deux lignes échangées contre deux autres, au même total
Les lignes	chaque table refaite contre celle d'avant, par `EXCEPT ALL` dans les deux sens	la moindre valeur qui diffère, exemplaire par exemplaire	ce qui était déjà faux avant
Les contrats	les 23 règles, sur tout l'historique	ce que la donnée promet, avant comme après	une promesse que personne n'a écrite

La première est celle du consommateur : ce que `LIVRAISON.md` annonce, table par table, et ce que le kit de départ d'Analytics Engineering vérifie. Le kit la passe sur votre propre projet, sans rien écrire : 22 tables et 580 907 lignes dans `raw`, 290 567 dans `argent`, douze contrôles, conforme, en deux secondes.

La deuxième est celle du Cours 06, reprise à l'échelle de la plateforme : 26 tables, la `RAW`, l'argent et le journal des chargements, comparées ligne pour ligne. Les deux sens comptent : ce qui manque au refait, et ce qu'il porte en trop. Seules les colonnes de l'heure du chargement sont écartées, parce qu'elles diffèrent par construction. Le journal, lui, se compare dans son état, sa dernière ligne par fichier, celle que le chargeur consulte ; ses versions remplacées se comptent à part (section 3). Résultat : 0 écart.

© 2026 iAlexMG. Tous droits réservés.

La troisième n'a pas besoin de l'entrepôt d'avant. Les contrats rejugent le refait sur tout l'historique et rendent les verdicts du Cours 11, avec la règle neuve à 0 : la commande relivrée, le concessionnaire en retard, le capteur figé, la pression instable, les dix heures hors plage. S'y ajoute la preuve du Cours 04, que la reconstruction rejoue par sa tâche `erp` : l'état tiré des extraits égale la base de l'ERP, 9 tables et 0 écart dans les deux sens.

Pourquoi — mettre de côté avant d'effacer. La deuxième preuve demande l'entrepôt d'avant, et il n'existe plus une fois effacé. On le déplace donc dans `plateforme/avant/` au lieu de le supprimer. Sans ce geste, il ne reste que les comptes et les contrats, et deux lignes échangées passeraient.

6. Le chemin chaud de bout en bout

Labo : `Labo_12_DE.md`, étape 7.

Un flux ne se reconstruit pas : il se rejoue. Le chemin chaud n'a pas de fichier à relire, il n'a que ce qui passe, et ce qui est passé ne repasse pas. Chez Boréal, le rejoueur du Cours 09 tient lieu d'usine : il republie octobre 2025 à 3 600 fois la vitesse réelle, et toute la chaîne se remet à travailler comme si les points arrivaient pour la première fois.

Repartir de zéro, pour le chemin chaud, veut dire vider les trois tables de `temps_reel` et laisser le reste en place. Les sujets de Kafka gardent leurs messages, et c'est voulu : les groupes de consommateurs ont retenu leur décalage au bout de chaque sujet (Cours 09), si bien qu'ils ne lisent que ce que le rejeu publie. Les messages d'avant restent relisibles par un groupe neuf, pour qui voudrait poser une autre question.

Le résultat se juge à un chiffre : l'alerte se lève le 2025-10-22 à 8 h 15, comme au Cours 09, à la minute près. Rien ne l'y oblige, sinon une propriété construite au Cours 09 : l'alerte juge le temps de l'événement, l'instant que porte chaque point, et non l'heure à laquelle il arrive. Le rejeu peut aller plus vite ou plus lentement, tourner un autre jour, sur une autre machine : l'alerte tombe au même instant de Boréal. Seule change `recu_le`, l'heure de réception, qui dit quand le rejeu est passé et rien d'autre.

7. Le tableau de bord, livrable versionné

Labo : `Labo_12_DE.md`, étape 8.

Le chemin chaud finit dans Grafana, et c'est le seul service que ce cours allume. Grafana lit la base de séries temporelles et dessine ce qu'on lui demande : la pression du réseau d'air, la vibration et la température du compresseur 2, et les alertes, en tableau et en marques sur les courbes.

Deux fichiers de la plateforme le branchent, et un fichier du dépôt le remplit.

Fichier	Où	Ce qu'il dit
`sources.yaml`	`plateforme/grafana/`, de l'équipe de la plateforme	la source de données : la base `usine` du service `series`, avec l'option TimescaleDB, sous l'identifiant `series`

© 2026 iAlexMG. Tous droits réservés.

Fichier	Où	Ce qu'il dit
`tableaux.yaml`	`plateforme/grafana/`	où chercher les tableaux de bord : `01_de/grafana`, à travers le projet monté, relu toutes les 30 secondes
`usine.json`	`01_de/grafana/`, votre dépôt	le tableau de bord : ses panneaux, leur requête SQL, la période, le fuseau

C'est la règle des DAG du Cours 06, appliquée à un tableau de bord : il vit dans le dépôt du code, avec son historique, et l'outil le lit sans le posséder. Un tableau de bord dessiné à la souris dans l'interface vit dans le volume du conteneur, et se perd avec lui. Écrit dans un fichier, il se relit, se compare, se reconstruit, et il revient à l'identique sur un Grafana neuf. Le réglage `allowUiUpdates: false` interdit d'enregistrer une retouche depuis l'interface : on le modifie dans son fichier, et Git garde la trace de chaque changement.

Le tableau de bord de ce cours est un tableau d'ingénieur. Il répond à la question de Data Engineering : « les données arrivent-elles, et que disent-elles ? ». Il ne répond pas à celle d'un directeur d'usine, qui voudrait peut-être une seule tuile rouge ou verte, ni à celle d'un technicien de maintenance, qui voudrait l'historique des bons de travail à côté de la vibration. Choisir ce qu'un lecteur voit d'abord, comment il le lit et ce qu'il en fait, c'est le métier de la formation Business Intelligence. Data Engineering monte la plomberie et garantit la donnée ; Business Intelligence en conçoit la lecture.

Piège — la période du tableau de bord. Grafana montre par défaut les six dernières heures, du temps de votre horloge. Les données de Boréal datent d'octobre 2025 : le tableau de bord s'ouvre vide, sans erreur. Le fichier fixe donc sa période sur le rejeu, du 13 octobre au 1er novembre 2025, et son fuseau sur celui de Boréal.

8. L'incident vu de l'amont

Labo : `Labo_12_DE.md`, étape 9.

Un incident traverse tout le parcours : la soupape d'un compresseur qui faiblit, un réseau d'air qui tremble, des colliers mal serrés, et des motoneiges qui surchauffent chez leurs clients trois mois plus tard. Chaque cours en a vu un morceau. Le capstone les remet dans l'ordre.

Date	Ce qui se passe	Où Data Engineering le voit
2025-10-15	la soupape du compresseur 2 commence à faiblir ; les motoneiges de la plage `MN26001238` à `MN26001466` passent au vissage jusqu'au 28	les points de l'historien, au bronze (Cours 08) ; `raw.erp_zserie`
2025-10-17	le nombre de points de pression double : 28 contre 12 d'habitude	la bande morte de l'historien (Cours 08 et 11)

© 2026 iAlexMG. Tous droits réservés.

Date	Ce qui se passe	Où Data Engineering le voit
2025-10-20	l'écart-type de la pression passe à 15,1 kPa	la règle `pression_instable` du contrat de l'historien, première alerte (Cours 11)
2025-10-22, 8 h 15	la vibration moyenne franchit 4,5 mm/s	l'alerte du chemin chaud, `temps_reel.alertes` (Cours 09)
2025-10-22, 9 h 12	un opérateur signale un bruit ; le bon `BT-2025-00739` se ferme sur « rien constaté »	la GMAO, `raw.usine_bons_travail` (Cours 08)
2025-10-27, 6 h 15	la vibration franchit 7,1 mm/s	le niveau d'arrêt de l'alerte
2025-10-28, 10 h 44	le compresseur 2 déclenche en pleine production	la GMAO, et la pression à 60,4 kPa d'écart-type
2025-12-24	premier code de surchauffe T310 d'une motoneige de la plage	`raw.telemetrie_sorties`, sous le nom neuf du champ (Cours 11)
janvier et février 2026	les réclamations du circuit de refroidissement montent	`raw.garanties_reclamations`
2026-02-16	le bulletin de service `BS-2026-01` demande d'inspecter les colliers de la plage	le corpus, déposé tel quel (Cours 03)

La lecture de ce tableau n'appartient pas à Data Engineering. Relier la vibration d'un compresseur aux réclamations de janvier demande de joindre l'historien, les numéros de série et les garanties, et la `RAW` ne joint rien (Cours 03). Ce que Data Engineering doit à l'analyse, ce sont trois faits livrés, chacun dans une table qu'une autre formation sait lire.

Fait livré	Où	Qui le lit
la date de l'alerte, 2025-10-22 à 8 h 15, six jours et deux heures avant le déclenchement	`temps_reel.alertes`, et le tableau de bord	Business Intelligence
le contrat qui a vu la pression instable, dès la journée du 20 octobre	`pipeline.controles`, règle `usine_historien.pression_instable`	Architecture et Gouvernance, qui en fait un engagement
une `RAW` où l'on retrouve la plage des motoneiges touchées	`raw.erp_zserie` : 229 motoneiges assemblées du 15 au 28 octobre sur `ASM-MN` ; leurs réclamations dans `raw.garanties_reclamations` ; le bulletin dans `raw.corpus_index`	Analytics Engineering, puis Data Science

© 2026 iAlexMG. Tous droits réservés.

Le troisième fait est le plus modeste et le plus important. Un analyste qui cherche pourquoi les réclamations montent en janvier doit trouver dans la `RAW`, sans rien demander à personne, le numéro de série de chaque motoneige, son poste et sa date d'assemblage, chaque réclamation dans chacun de ses états, et le bulletin qui nomme la plage. Dans la `RAW` refaite, la plage ouvre 69 réclamations de décembre à mars, dont 36 au circuit de refroidissement : 1 en décembre, 15 en janvier, 15 en février, 5 en mars, après le bulletin. Rien de cela n'a été ajouté pour l'incident : c'est la règle du Cours 03, qui garde tout ce qui arrive, tel quel, avec son lignage.

9. La passation

Labo : `Labo_12_DE.md`, étapes 10 et 11.

Une plateforme se passe par écrit. Quatre documents s'en chargent, et chacun a son lecteur.

Document	Son lecteur	Ce qu'il dit
le `README.md` du dépôt	l'ingénieur qui reprendra le code	ce qu'est le dépôt, ce qu'il suppose, les commandes dans l'ordre, ce qui n'y est pas et pourquoi, comment refaire et comment prouver
`LIVRAISON.md`	les formations qui consomment	chaque produit, ses tables, leurs comptes, ce qu'un consommateur doit savoir, la commande qui le refait
les contrats	le producteur et le consommateur, et une machine	ce qui est promis, sous une forme qui se vérifie à chaque livraison
le kit 02	Analytics Engineering, qui arrive sans la plateforme de Data Engineering	la `RAW` et l'argent refaits en deux minutes, sans Airflow ni Docker, et comparés à `LIVRAISON.md`

Le `README.md` est celui qu'on oublie, parce que celui qui l'écrit n'en a pas besoin. Un bon `README.md` de dépôt de données ne raconte pas le code : il dit ce qu'il faut avoir avant de commencer, la première commande à taper et ce qu'elle doit afficher, la seconde, et ainsi de suite, jusqu'à la preuve. Il dit aussi ce qui n'est pas dans le dépôt, les données, les secrets, la plateforme, et où le trouver. Le test est simple : une personne qui ne vous connaît pas refait la plateforme en suivant le fichier, sans vous appeler.

Reste à marquer l'état livré. Une étiquette Git, `v1.0`, nomme un point de l'historique que d'autres peuvent citer : le kit, `LIVRAISON.md` et les formations suivantes s'y réfèrent, et la branche principale peut continuer d'avancer sans que personne ne lise un état à moitié fini. La numérotation est celle des contrats (Cours 11), appliquée au dépôt entier : un changement qui casse un consommateur fera la `v2.0`.

Snowflake. Deux gestes du capstone ont leur équivalent direct dans Snowflake. Mettre de côté avant d'effacer : `CREATE DATABASE boreal_avant CLONE boreal;` fait une copie sans copie, qui partage les micro-partitions de l'original et ne coûte que ce qui change ensuite ; avec le voyage dans le temps, `CLONE boreal AT (OFFSET => -3600)` clone l'état d'il y a une heure. Et orchestrer une reconstruction : un graphe de tâches (*task graph*), dont chaque tâche nomme

© 2026 iAlexMG. Tous droits réservés.

ses prédécesseurs par `AFTER`, se lance à la main par `EXECUTE TASK` sur sa tâche racine, et peut finir par une tâche de clôture, `FINALIZE`, qui tourne même quand une autre a échoué. D'après la documentation de Snowflake, consultée le 2026-09-23 ; la plateforme appartient à la formation Snowflake.

Databricks. Le DAG de reconstruction s'écrirait en tâche Lakeflow (*Lakeflow Jobs*) : des tâches reliées par leurs dépendances, sans déclencheur, lancées à la main. Le tableau de bord se ferait en tableau de bord AI/BI, qui se range comme un fichier dans un dossier Git de l'espace de travail et se déploie, avec les tâches, par les Declarative Automation Bundles : la même règle qu'ici, un livrable qui vit dans un dépôt plutôt que dans une interface. D'après la documentation de Databricks, consultée le 2026-09-23.

Recoupements — ce que ce cours partage avec ses voisines.

- Analytics Engineering : la `RAW` refaite par le capstone est ce que le kit 02 livre à AE, et `LIVRAISON.md` le décrit. Le `STAGING` d'AE commence où ce cours s'arrête, et c'est lui qui joindra les numéros de série aux réclamations.
- Business Intelligence : le tableau de bord Grafana passe à BI, qui en conçoit la lecture, les seuils visibles, le public et l'ordre des panneaux. Ici, on en monte la plomberie et on garantit la donnée qu'il lit.
- Data Science : les séries de l'usine, refaites et prouvées, nourrissent la maintenance prédictive du compresseur et la prévision de la demande électrique. Ce cours garantit qu'elles reviennent identiques ; DS en fait des modèles.

Points clés à retenir

- Une plateforme est finie quand elle se refait de zéro, qu'une preuve dit qu'elle est revenue identique, et qu'une autre personne peut la reprendre.
- « De zéro » veut dire à partir du dépôt, des livraisons gardées et de la plateforme. Qui ne garde pas ses livraisons ne peut que restaurer une sauvegarde.
- Ce qui s'écrit se versionne ; ce qui se dérive se refait ; les secrets ne voyagent jamais.
- Une reconstruction refait un état, jamais une histoire : ce qu'elle ne peut pas refaire se compte à part, et se garde par une sauvegarde.
- Les flèches d'un DAG disent les dépendances des données ; le pool dit la ressource partagée.
- Changer de machine fait tomber ce qu'aucun test ne voit : un fichier attaché deux fois, un pilote absent de l'image, un fuseau horaire différent.
- Refaire n'est pas prouver : les comptes, puis les lignes dans les deux sens, puis les contrats.
- Un flux ne se reconstruit pas, il se rejoue ; l'alerte revient à la même minute parce qu'elle juge le temps de l'événement.
- Un tableau de bord est un fichier du dépôt. Data Engineering en monte la plomberie ; Business Intelligence en conçoit la lecture.
- La `RAW` doit porter de quoi remonter un incident sans rien demander à personne.
- Une plateforme se passe par écrit : le `README.md` du dépôt, `LIVRAISON.md`, les contrats, le kit, et une étiquette qui fixe l'état livré.

© 2026 iAlexMG. Tous droits réservés.

Exercices

Les énoncés et leur corrigé sont dans `Exercices_corriges_12_DE.md`. Faites le labo avant : plusieurs questions portent sur la plateforme que vous aurez refaite vous-même.

© 2026 iAlexMG. Tous droits réservés.